Vision with neural networks

Ulf Krumnack Robin Rawiel

Vision has always been one of the central ambitions of artificial intelligence. From a cognitive science perspective, this is not surprising: visual perception is one of the most important ways in which biological systems acquire information about the world. If we want to build systems that can recognize objects, navigate environments, or interpret scenes, then learning to process images is a natural and important challenge. At the same time, vision is difficult. Images are high-dimensional, structured, and highly variable. The same object can appear in different positions, scales, lighting conditions, and backgrounds, and yet we still recognize it as the same object with little effort.

Computer vision also played a special role in the rise of modern deep learning. While neural networks had existed for decades, it was large-scale image recognition that provided some of the most visible early successes of deep architectures. In particular, the breakthrough results on the ImageNet benchmark showed that convolutional neural networks could learn powerful visual representations directly from data and outperform earlier hand-engineered approaches by a large margin. This helped convince both researchers and the broader public that deep learning was not only an elegant idea, but also a practically effective one. In this chapter, we take a first step into this area. We will see how images can be represented mathematically, why convolution is such a natural operation for visual data, and how convolutional neural networks exploit the spatial structure of images in a way that ordinary multilayer perceptrons do not.

1 Images as tensors

To understand convolutional neural networks, we must first answer a simple question: **what is an image, mathematically?** In everyday life, an image is something we look at. In deep learning, however, an image is a numerical object. More precisely, it is a structured collection of numbers arranged on a grid. This grid structure is crucial, because nearby pixels are usually related to each other, and later convolution will make direct use of this fact.

A digital image consists of many small picture elements called pixels. These pixels are arranged on a rectangular grid. In a grayscale image, each pixel stores only a single intensity value, for example how dark or bright that location is. We can therefore represent such an image as a matrix

pixels

grayscale image

$$\mathbf{X} \in \mathbb{R}^{H \times W},$$

where H denotes the height of the image, that is, the number of rows, and W denotes the width, that is, the number of columns. The entry $\mathbf{X}[i, j]$ then refers to the pixel value at row i and column j.

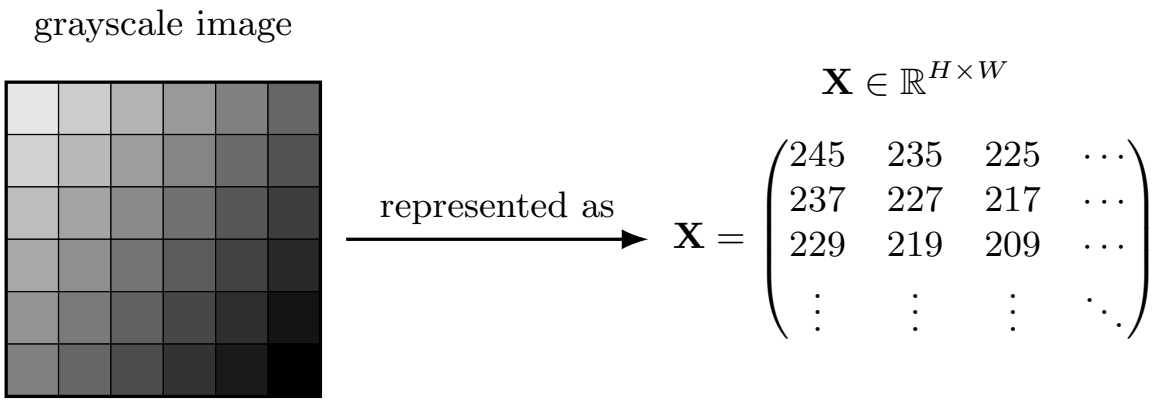


Figure 1: A grayscale image as a matrix. A grayscale image can be represented as a matrix $\mathbf{X} \in \mathbb{R}^{H \times W}$, where each entry stores the intensity value of one pixel.

Pixel values themselves can be stored in different ways. In many image files, they are integers between 0 and 255. In machine learning, it is common to rescale such values to real numbers in the interval $[0, 1]$ or sometimes to values centered around zero. This preprocessing does not change the visual content of the image, but it makes numerical optimization more stable and convenient.

A color image contains more information. Instead of storing only one intensity value per pixel, it stores several values, one for each color channel. In the standard RGB representation, these channels correspond to red, green, and blue. Each channel is itself a two-dimensional grid, so a color image is naturally represented not by a matrix, but by a tensor. In this course, we use the term tensor in a simple sense: a tensor is a multidimensional array of numbers. Using a channel-first convention, we write a color image as

$$\mathbf{X} \in \mathbb{R}^{C \times H \times W},$$

where C is the number of channels. For an RGB image, we usually have $C = 3$. Some software libraries instead use a channel-last convention,

$$\mathbf{X} \in \mathbb{R}^{H \times W \times C},$$

so it is always important to check which convention is being used.[1]

At this point, one small indexing issue is worth making explicit. In geometry, people often think in coordinates (x, y), where x runs horizontally and y vertically. In arrays, however,

[1] Historically, traditional computer vision often used the channel-last convention, and many classical computer vision libraries still do so. In deep learning, both conventions are common. PyTorch, Theano, and MXNet typically use channel-first layouts, while TensorFlow and Keras long preferred channel-last. Hardware considerations also play a role: TPUs often prefer channel-last layouts such as NHWC because of their memory architecture, while GPUs traditionally favored channel-first layouts such as NCHW, although modern GPUs also support fast channel-last computation. In this lecture we use PyTorch and therefore stick to the channel-first convention.

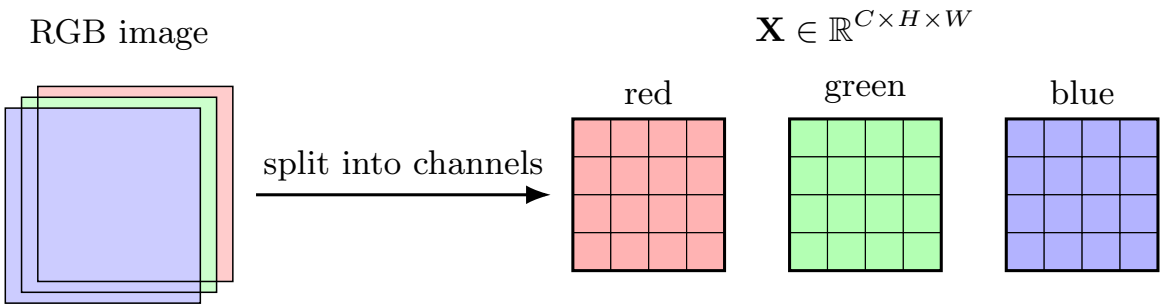


Figure 2: A color image as a tensor of channels. An RGB image is naturally represented as a tensor with three channels, one each for red, green, and blue.

indexing usually goes by row first and column second. This means that the first spatial index refers to the vertical position, and the second to the horizontal position. In image processing, coordinates are often counted from the top-left corner. This may feel slightly unusual at first, but it is the convention used in many practical settings, and it will matter once we start writing down convolution formulas.

This description also helps us connect computer vision to the previous chapter. In an MLP, the input was typically a vector. Of course, we could flatten an image into a long vector and feed it into an MLP. But doing so would throw away the explicit spatial organization of the image: pixels that are neighbors in the two-dimensional grid would no longer be represented as neighbors in any visible way. Convolutional neural networks are designed to avoid this loss of structure. They work directly with the tensor form of the image and thereby preserve the spatial relationships between pixels.

Before turning to large modern datasets, it is helpful to mention two classical benchmarks that played an important role in the history of deep learning. One of the most famous is the MNIST dataset, a collection of handwritten digit images. Each image shows one digit from 0 to 9 and is stored as a small grayscale image of size 28×28. The dataset contains 60,000 training images and 10,000 test images. Because the images are simple and the task is well defined, MNIST became a standard introductory benchmark for image classification. Even today, it is often used for first experiments with neural networks, although by modern standards it is considered too easy to be a challenging vision benchmark.

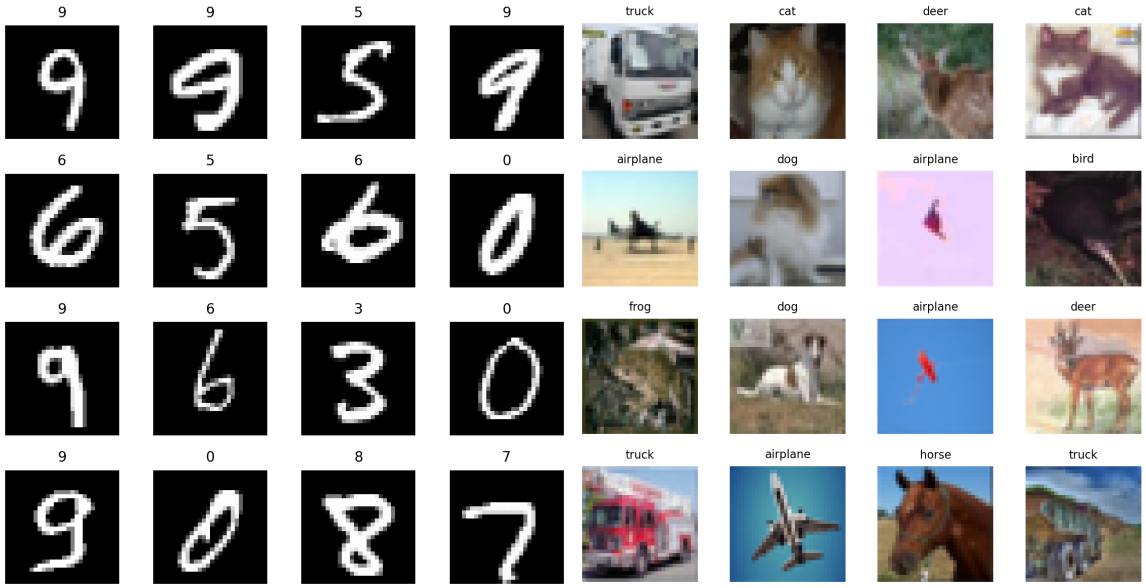
MNIST

A slightly more realistic benchmark is the CIFAR-10 dataset. It consists of small color images of size 32×32 drawn from 10 object categories, including airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks. CIFAR-10 contains 50,000 training images and 10,000 test images. Compared with MNIST, it is more difficult because the images are colored, the objects vary more strongly in appearance, and the backgrounds are more complex. CIFAR-10 therefore became an important intermediate benchmark: still small enough for efficient experiments, but already rich enough to make convolutional architectures genuinely useful.

CIFAR-10

A famous example of image data at large scale is the ImageNet dataset. ImageNet was built by combining ideas from computer vision with the semantic category structure of WordNet. The best-known benchmark based on it, ImageNet 2012 (also called ILSVRC2012),

ImageNet



(a) MNIST (b) CIFAR-10

Figure 3: Examples of two classical vision benchmarks. MNIST consists of grayscale handwritten digits, while CIFAR-10 contains color object images from ten categories.

became the gold standard for object recognition and is often seen as one of the landmarks that triggered the modern deep learning revolution. The benchmark contains about 1.28 million training images and 50,000 validation images, organized into 1,000 object categories. These categories cover a wide range of concepts, including animals, plants, vehicles, tools, appliances, furniture, musical instruments, food, and many other everyday objects. The dataset is not perfectly balanced in a semantic sense; for example, more than 100 classes are dedicated to dog breeds. Its historical importance comes largely from the success of **AlexNet**, a deep convolutional neural network (CNN) that won the 2012 competition by a large margin and demonstrated that learned visual features could outperform earlier hand-engineered approaches. Because of this history, many powerful pretrained CNN models are available today, often trained on ImageNet and then adapted to new tasks.

Once we represent an image as a grid of values, the next question is how to extract useful visual structure from this grid. A central operation for this is convolution.

2 Convolution: local feature detection

Images are not just large collections of unrelated numbers. Their most important structure is local: neighboring pixels usually belong to the same object, the same texture, or the same edge. For this reason, image analysis often begins by looking not at the whole image at once, but at small local neighborhoods. The central operation that does exactly this is convolution.

convolution

The basic idea is simple. We choose a small matrix of numbers, called a kernel or filter, and move it across the image. At each position, we compare the kernel to the corresponding image patch. If the local patch resembles the pattern encoded by the kernel, the response will be strong; if it does not, the response will be weak. In this sense, a convolution kernel acts like a small pattern detector.

Let us first consider a grayscale image

$$\mathbf{X} \in \mathbb{R}^{H \times W}$$

and a kernel

$$\mathbf{K} \in \mathbb{R}^{k_h \times k_w}.$$

When we place the kernel on a local patch of the image, we multiply corresponding entries and sum the results. The resulting number becomes one entry of the output image, often called a feature map. Written formally, this can be expressed as

$$Z_{[i,j]} = \sum_{u,v} K_{[u,v]} \mathbf{X}_{[i+u,j+v]}.$$

You should read this formula as a local matching rule: take the patch around position (i, j), compare it with the kernel by weighted multiplication, and record the sum. Deep learning libraries often implement a very closely related operation called *cross-correlation* rather than strict mathematical convolution, but for our purposes the intuition is the same: a small filter is slid across the image to measure how strongly a local pattern is present.

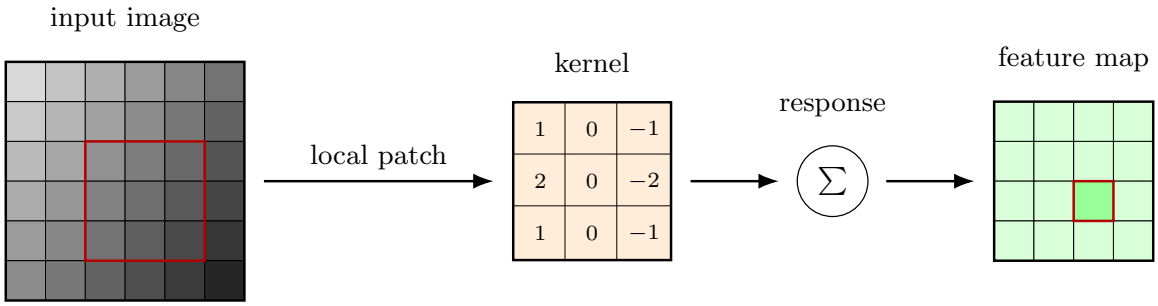


Figure 4: Convolution as local pattern matching. A small kernel is applied to a local image patch by multiplying corresponding entries and summing the results. Repeating this across the image produces a feature map.

This becomes most intuitive when we look at concrete examples. In classical computer vision, many useful kernels were designed by hand. A famous example is an edge detector such as a Sobel filter. Such a kernel gives a strong response when the image intensity changes sharply in one direction, for example from dark to bright across neighboring pixels. Since edges are often associated with object boundaries, such kernels were historically very important. Another example is a smoothing or blur kernel, which averages nearby pixels. This suppresses small random fluctuations and can therefore reduce noise. In general, different kernels highlight different kinds of local structure: edges, corners, texture changes, or smooth regions.

The output of a convolution is therefore another image-like array, but its meaning is different from that of the original image. It no longer represents raw pixel intensity. Instead, it represents the strength of a particular detected feature at each spatial location. High values indicate that the local patch matches the filter well; low values indicate a poor match. This is why the result is called a feature map: it tells us where in the image a certain feature appears.

So far, this description has assumed a single input channel. Real color images, however, usually have several channels. In our notation, an input image is a tensor

$$\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}.$$

A convolution filter must therefore also extend across all input channels. One filter has shape

$$C_{\text{in}} \times k_h \times k_w.$$

This is an important point: a filter does **not** operate on each channel independently and then keep them separate. Instead, it contains one small kernel for each input channel, computes a response in each channel, and then sums these responses to produce a single output feature map. Thus one multi-channel filter maps many input channels to one output channel.

This may sound abstract at first, but the intuition is quite natural. A grayscale edge detector responds only to intensity patterns. A multi-channel filter can do more: it can respond to color-specific patterns as well. For example, it could detect a red-green contrast, a blue-yellow boundary, or an edge that is strong in one color channel but weak in another. In this way, convolution on color images can capture both spatial structure and color structure at the same time.

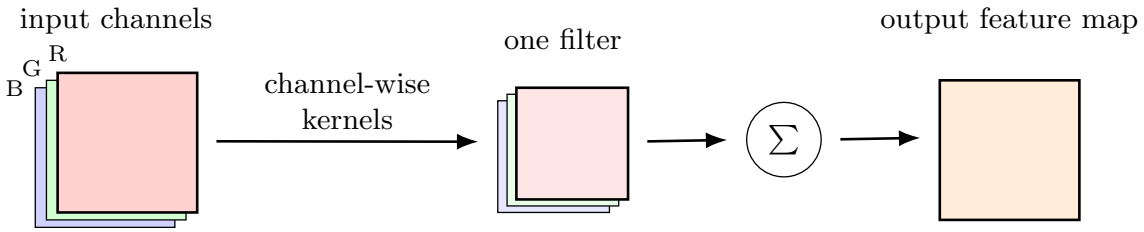


Figure 5: A convolution filter spans all input channels. For a color image, one filter contains one small kernel per input channel. The channel-wise responses are summed to produce one output feature map.

The central message of this section is therefore simple but powerful: convolution is a way of detecting local patterns by repeatedly applying the same small filter across an image. Classical computer vision often designed such kernels by hand. Convolutional neural networks keep the same basic idea, but instead of fixing the kernels manually, they learn useful filters directly from data.

3 Convolutional layers: learned kernels and shared weights

In the previous section, convolution appeared as a hand-designed operation: we chose a kernel, slid it across the image, and interpreted the resulting feature map. Convolutional neural networks keep exactly this local pattern-matching idea, but with one crucial difference: the filters are no longer designed by hand. Instead, their entries are treated as trainable parameters and are learned from data. During training, backpropagation adjusts these kernel values so that the resulting feature maps become useful for the task at hand.

trainable parameters

A convolutional layer therefore acts as a learned feature extractor. It takes an input tensor

convolutional layer

$$\mathbf{X} \in \mathbb{R}^{C_{\text{in}} \times H \times W}$$

and produces an output tensor

$$\mathbf{Z} \in \mathbb{R}^{C_{\text{out}} \times H' \times W'},$$

where each of the C_{out} output channels is generated by one learned filter. Each filter spans all input channels, so its shape is

$$C_{\text{in}} \times k_h \times k_w.$$

Applying one such filter across the full spatial extent of the image produces one output feature map. Using many different filters in parallel produces many output channels, each highlighting a different kind of pattern.

Two ideas make convolutional layers especially well suited for images. The first is local connectivity. In a fully connected layer, every hidden unit depends on every input value. If the input is an image, this means that even a unit responsible for detecting a tiny local pattern would still be connected to every pixel in the image. A convolutional layer works differently: a unit at a given spatial location only depends on a small local patch of the input. This matches the structure of images much better, because nearby pixels are much more strongly related than pixels that are far apart.

local connectivity

The second idea is shared weights. The same filter is reused at every spatial position. In other words, the local detector that looks for a certain edge, corner, or texture in one part of the image is also used in every other part of the image. This is a very strong and very useful assumption. If a vertical edge is meaningful near the top-left corner of an image, it is likely meaningful near the bottom-right corner as well. Weight sharing therefore gives the model a form of translation-related robustness: it can recognize the same local pattern regardless of where it appears.

shared weights

Weight sharing is also the main reason why convolutional layers are so parameter-efficient. Suppose we have a convolutional layer with kernel height k_h, kernel width k_w, C_{in} input channels, and C_{out} output channels. Then each output channel has one filter bank of size

$$C_{\text{in}} \times k_h \times k_w,$$

which contributes $k_h k_w C_{\text{in}}$ weights. If we have C_{out} such filters, this gives

$$k_h k_w C_{\text{in}} C_{\text{out}}$$

weight parameters in total. If each output channel also has its own bias, we obtain

$$k_h k_w C_{\text{in}} C_{\text{out}} + C_{\text{out}}$$

trainable parameters.

This is dramatically fewer than in a fully connected layer. Consider a color image of shape

$$3 \times 32 \times 32.$$

Flattening it gives

$$3 \cdot 32 \cdot 32 = 3072$$

input values. A fully connected layer with 100 hidden units would therefore require

$$3072 \cdot 100 + 100 = 307300$$

parameters. By contrast, a convolutional layer with 16 output channels and 3×3 kernels needs only

$$3 \cdot 3 \cdot 3 \cdot 16 + 16 = 448$$

parameters. This difference is enormous. The convolutional layer is not only more structured, but also far more economical. It uses many fewer parameters while still preserving the spatial organization of the image.

This comparison also reveals why CNNs scale better to image data than plain MLPs. If we flatten an image, we lose the explicit two-dimensional structure and force the network to learn everything from scratch with a huge number of independent weights. A convolutional layer instead builds in the assumption that visual patterns are local and reusable across space. This inductive bias is exactly what makes convolutional networks so effective in computer vision.

At the same time, a convolutional layer does not simply keep the input shape unchanged. In general, it maps

$$\mathbb{R}^{C_{\text{in}} \times H \times W} \to \mathbb{R}^{C_{\text{out}} \times H' \times W'}.$$

The number of channels changes because we apply several learned filters, and the spatial dimensions may also change. How exactly H' and W' are determined depends on additional architectural choices such as padding and stride. These choices, together with pooling and related design decisions, are the next ingredients we need in order to build full convolutional networks.

4 Building CNNs: padding, stride, pooling, and global pooling

A single convolutional layer is already a useful learned feature detector, but practical convolutional neural networks need a few more architectural ingredients. Once we start stacking many layers, several questions immediately arise. What should happen at the image boundary, where a kernel no longer fully fits? How do we control the spatial size of the output? How can we gradually reduce resolution while keeping the most important

information? And near the end of the network, how do we convert many spatial feature maps into a compact representation that can be used for classification? The standard answers to these questions are padding, stride, pooling, and global pooling.

The first issue is the boundary problem. A convolution kernel can only be placed where all of its entries overlap with valid image pixels. If we use no special treatment at the border, then some positions near the edge are excluded. As a result, the output feature map becomes smaller than the input. This is sometimes perfectly fine, but sometimes we would prefer to preserve the spatial size. A common solution is padding: we add extra values around the border of the input, usually zeros, before applying the convolution. With enough padding, the kernel can also be centered near the image edges. Two common terms are worth knowing. A valid convolution uses no padding at all, so the output shrinks. A same padding chooses the padding so that the output has roughly the same spatial dimensions as the input, at least when the stride is 1.

padding

valid convolution

same padding

A second important mechanism is stride. The stride tells us how far the kernel moves each time it is shifted across the image. With stride 1, the kernel moves one pixel at a time, producing a densely sampled feature map. With stride 2, it jumps by two pixels, so fewer output positions are computed and the spatial dimensions become smaller. In this way, stride provides a simple form of downsampling. Compared with stride 1, stride 2 reduces computational cost and enlarges the region of the input that later units can indirectly depend on.

stride

Another common way to reduce spatial resolution is pooling. Pooling does not use learned weights. Instead, it summarizes a small neighborhood by a fixed rule. The most common variant is max pooling. For example, in a 2×2 max-pooling operation, we look at each 2×2 patch and keep only its largest value. This makes the representation smaller, but it keeps strong activations that often correspond to the presence of an important local feature. Intuitively, pooling says that the exact position of a feature within a very small neighborhood may not matter much; it is often enough to know that the feature is present somewhere nearby. For this reason, pooling introduces a limited form of local invariance.

pooling

max pooling

Near the end of a CNN, we often want to aggregate information across the whole image. A particularly elegant method is global average pooling. Here, each feature map is averaged over its entire spatial extent, so that one output channel becomes a single number. If the network has C channels in its final convolutional layer, global average pooling produces a vector in $\mathbb{R}^C$. This creates a compact summary of "how strongly each feature was present somewhere in the image" and often replaces the large fully connected heads that were common in older architectures.

global average pooling

Putting these ingredients together gives a typical CNN pipeline. We repeatedly apply convolutions, nonlinearities, and sometimes pooling. As we go deeper into the network, the spatial resolution often decreases, while the number of channels often increases. The idea is that early layers describe the image in a fine-grained local way, whereas later layers trade spatial precision for richer and more abstract feature descriptions. A useful concept here is the receptive field. The receptive field of a unit is the region of the original input image that can influence its value. In early layers, receptive fields are small, because each unit only sees a small local patch. In deeper layers, receptive fields grow, because each layer builds on

receptive field

outputs from previous layers. As a result, later units can depend on much larger parts of the input image and can therefore encode more global structure.

Historically, these ideas became especially visible in influential architectures such as **AlexNet** and **VGG**. AlexNet was the landmark CNN that achieved a breakthrough on ImageNet and helped convince the field that deep learned visual features could outperform hand-engineered methods. VGG followed with a particularly simple but powerful design principle: instead of using large convolutions, it stacked many small 3×3 convolutional layers. This emphasized that depth itself can be a major source of expressive power, even when the individual building blocks remain simple.

By now we have the main ingredients of a convolutional network: local convolutions, shared weights, resolution control through padding and stride, and spatial summarization through pooling. As architectures became deeper, however, a new problem emerged. Even though deeper networks were often more expressive, they also became harder to train effectively.

5 From deep CNNs to residual networks

So far, depth has appeared as a clear advantage. Earlier in this course we argued that deeper networks can build hierarchical representations more efficiently than shallow ones. In computer vision, this idea is especially natural. Early layers may detect simple patterns such as edges and small texture changes. Later layers can combine these into motifs, object parts, and eventually whole objects. This is one of the main reasons why deeper CNNs often perform better than very shallow ones.

However, there is an important catch: making a network deeper does not automatically make it easier to train. In principle, additional layers increase expressive power. In practice, optimization can become much more difficult. One of the best-known reasons is the vanishing gradient problem. During backpropagation, gradient information is passed backward through many layers by repeatedly multiplying local derivative factors. If many of these factors are smaller than 1, the gradient can shrink very quickly as it moves toward earlier layers. Then the first layers receive only tiny updates and learn very slowly. In extreme cases, they may hardly learn at all.

vanishing gradient

The practical symptom is frustrating but important: a deeper plain network can sometimes train worse than a shallower one. One might expect that adding more layers should at least preserve performance, since the extra layers could in principle learn to do nothing useful. But in reality, optimization often gets in the way. The deeper model may be harder to fit, even though it is more expressive. This observation helped motivate one of the most influential ideas in modern deep learning: the residual connection.

residual connection

The residual idea starts from a simple reformulation. Suppose a block of layers is meant to learn some mapping $H(\mathbf{x})$. Instead of learning this full mapping directly, we ask the block to learn only the difference between the desired output and the input:

$$F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}.$$

Then we can rewrite the target mapping as

$$H(\mathbf{x}) = F(\mathbf{x}) + \mathbf{x}.$$

This may look like a trivial algebraic step, but it has an important practical consequence. If the best behavior of the block is close to simply passing its input through unchanged, then the residual function F only has to learn a small correction, possibly even something close to zero. That is often much easier than forcing the whole block to represent the identity transformation implicitly through many parameters.

A residual block therefore contains not only a usual stack of layers, but also a shortcut connection that skips over them. If the input is $\mathbf{x}$, the block outputs

residual block
shortcut connection

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x}.$$

The shortcut gives information a direct path through the network. This helps in two ways. First, it makes it easier for the block to behave like the identity map when that is useful. Second, it gives gradients a more direct route backward through the network, which helps reduce optimization difficulties in very deep models.

Another architectural ingredient that became standard in such networks is batch normalization. The core idea of batch normalization is to normalize layer activations so that their mean and variance stay in a more controlled range during training. This often stabilizes optimization, allows for larger learning rates, and helps mitigate vanishing or exploding gradients. In standard CNN practice, batch normalization is usually placed after a convolution and before the activation function. In the residual networks used in practice, each important convolution inside a block is typically followed by a batch normalization layer. We will not go into all mathematical details here, but it is useful to know that modern deep CNNs rely not only on convolutions and skip connections, but also on such normalization techniques.

batch normalization

A ResNet is a network built by stacking many residual blocks. This simple idea had a huge impact. It made very deep networks much easier to train and enabled architectures with dozens or even hundreds of layers. Residual connections were first developed in computer vision, but the idea has since spread far beyond CNNs and now appears throughout deep learning, including sequence models and transformers. In that sense, ResNets are not just a successful image architecture; they introduced a general design principle for making deep models trainable.

ResNet

This brings us to the end of our first look at convolutional neural networks. We began by representing images as tensors, introduced convolution as local pattern detection, saw how convolutional layers learn reusable filters with shared weights, and then added the practical ingredients that make full CNNs work. Residual networks show that architectural design matters not only for representation, but also for optimization. So far, gradients have been used to adapt the parameters of a network. But gradients can also be used in a surprising different way: to adapt the input itself.

6 Excursion: Adversarial examples

So far, whenever we discussed gradient-based learning, the object we modified was the parameter vector of the model. The input $\mathbf{x}$ was considered fixed, the loss $\mathcal{L}$ was computed from that input and its label, and the gradient told us how to change the parameters in order to improve the model. But mathematically there is nothing special about the parameters in this regard. The loss is also a function of the *input*. This means that we can ask a different question: instead of changing the model so that it classifies the input better, what happens if we keep the model fixed and change the input itself?

adversarial examples

This idea leads to the phenomenon of adversarial examples. An adversarial example is an input that has been modified in a carefully chosen way so that the model makes a wrong prediction, even though the modification may be very small and sometimes hardly visible to humans. In the case of images, this means that a classifier may assign a confident but incorrect label to an image that looks, to us, almost unchanged. This is surprising at first, but from the perspective of optimization it is a very natural possibility. If the loss depends on the input, then the gradient with respect to the input tells us in which direction we should change the image to increase or decrease that loss most strongly.

Suppose we have a trained classifier f_θ, an input image $\mathbf{x}$, and a label y. In the usual training setting, we would compute the gradient

$$\nabla_\theta \mathcal{L}(f_\theta(\mathbf{x}), y)$$

and use it to update θ. For adversarial attacks, the parameters remain fixed and we instead compute

$$\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y).$$

This gradient tells us how sensitive the loss is to small changes in the input image itself. If our goal is to make the classifier perform worse on the current image, then we want to *increase* the loss. A particularly simple method for doing this is the Fast Gradient Sign Method (FGSM). It constructs an adversarial example by taking a small step in the direction of the sign of the input gradient:

Fast Gradient Sign Method (FGSM)

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \epsilon \, \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y)) \, .$$

Here $\epsilon > 0$ controls how large the perturbation is. The sign function is applied elementwise, so each input component is nudged either upward or downward. The use of the sign has a simple intuition: it changes each pixel in the direction that increases the loss, without worrying about the exact magnitude of the gradient at that pixel. This gives a perturbation that is easy to compute and often surprisingly effective.

one-step attack

iterative attack

The basic FGSM attack is a one-step attack, because the adversarial example is produced in a single update. One can also repeat the same idea several times using smaller steps. Starting from the original image, we repeatedly compute the gradient with respect to the current perturbed input and then apply another small sign step. This yields an iterative attack. Intuitively, the one-step method takes one jump in a harmful direction, whereas the iterative method repeatedly refines the attack. Because repeated updates can otherwise move

the image too far away from the original, it is common to restrict the perturbation so that each pixel remains within a fixed distance of the original image. In practice this is usually done by *clamping* the perturbed image after each step. Iterative attacks are often stronger than one-step attacks because they can follow the local geometry of the loss landscape more carefully.

Up to this point, the goal was only to make the classifier fail. This is called an untargeted attack: any wrong prediction counts as success. But we can be more specific. Suppose we want the network not merely to be wrong, but to classify the image as some chosen target class y_{target}. Then we define the loss with respect to that target label and perturb the image so that the target loss becomes *smaller*. In other words, instead of pushing the image away from its current class, we pull it toward the target class. A one-step targeted method therefore takes the form

untargeted attack

$$\mathbf{x}_{\text{adv}} = \mathbf{x} - \epsilon \, \text{sign}\big(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y_{\text{target}})\big) \, .$$

The minus sign is crucial. In the untargeted case, we wanted to increase the loss for the original label, so we moved in the positive gradient direction. In the targeted case, we want to decrease the loss for the chosen target class, so we move in the negative gradient direction. Just as before, this targeted method also has an iterative version, where many small steps are taken and the perturbation is restricted after each step.

Adversarial examples illustrate two important lessons. First, they show very clearly that backpropagation is not tied to parameter learning. The same machinery can be applied to any quantity that influences the loss, including the input. Second, they reveal that highly accurate classifiers may still rely on decision boundaries that are unexpectedly fragile. A tiny change in a high-dimensional input can be enough to cross such a boundary and trigger a very different prediction. In this sense, adversarial examples are not merely a technical curiosity. They provide insight into the geometry of learned representations and remind us that good performance on standard test data does not automatically imply robustness.

7 Summary and outlook

In this chapter, we moved from general multilayer perceptrons to architectures that are specifically designed for image data. The key step was to recognize that images are not naturally just long vectors. They are structured objects with spatial dimensions and, in the color case, multiple channels. Representing images as tensors allows us to preserve this structure and build models that can use it directly.

We then introduced convolution as a method for local pattern detection. A small filter is moved across the image, and the resulting responses form a feature map. This gave us the basic intuition behind convolutional neural networks: instead of hand-crafting filters, as classical computer vision often did, CNNs learn useful filters from data. The defining ideas of local connectivity and shared weights make this approach especially well suited for images and far more parameter-efficient than fully connected layers.

Building practical CNNs required a few further ingredients. We discussed how padding handles image boundaries, how stride controls spatial resolution, how pooling summarizes local neighborhoods, and how global average pooling can compress a stack of feature maps into a compact representation near the end of the network. Together, these mechanisms form the standard building blocks of modern vision architectures.

We also saw that depth is both a blessing and a challenge. Deeper CNNs can build more abstract visual representations, but they are also harder to optimize. The vanishing gradient problem can make very deep plain networks difficult to train. This motivated the idea of residual connections, which led to ResNets and made very deep architectures much more practical. Finally, the excursion on adversarial examples showed that gradients do not only tell us how to change parameters. They can also tell us how to change the input itself, sometimes in ways that are surprisingly revealing about the learned model.

General Concept: Chapter takeaway

A convolutional neural network uses local, shared filters to process images in a way that respects spatial structure. This makes CNNs both parameter-efficient and well suited for hierarchical visual feature learning.

convolutional neural network

This was the fourth chapter of the first part of the lecture, which focuses on supervised learning. One final chapter remains in this part. There we will step back, recap the most important ideas developed so far, and discuss several practical details that we have only touched on briefly, such as optimizers, activation functions, preprocessing and data pipelines, and a few further aspects of optimization.

References and further reading

The ideas in this chapter connect modern deep learning with a long tradition in computer vision. For a broad and readable historical overview of computer vision as a field, *Computer Vision: Models, Learning, and Inference* by Prince provides a very accessible entry point (Prince 2012). It helps place convolutional networks into the larger context of image representations, filtering, and recognition.

A key historical milestone for convolutional neural networks is the work of LeCun and colleagues on gradient-based learning for document recognition (LeCun et al. 1998). This paper introduced and popularized early CNN ideas on tasks such as handwritten digit recognition and showed that learned local filters can outperform more hand-engineered approaches. Another landmark is the AlexNet paper by Krizhevsky, Sutskever, and Hinton (Krizhevsky et al. 2012), which demonstrated the power of deep CNNs on the ImageNet benchmark and is widely seen as one of the events that triggered the modern deep learning boom in computer vision.

For the problem of training very deep networks, the ResNet paper by He and colleagues is the central reference (He et al. 2016). It introduced residual connections and showed that

these make much deeper CNNs trainable in practice. Since then, residual architectures have influenced not only vision models, but deep learning more generally.

For textbook treatment, *Deep Learning* by Goodfellow, Bengio, and Courville contains a concise introduction to convolution, pooling, and convolutional networks (Goodfellow et al. 2016). For those who would like a more applied computer vision perspective, *Dive into Deep Learning* by Zhang et al. is also a useful resource, especially because it combines explanations with executable code (Zhang et al. 2023). For a broader modern introduction to visual recognition, *Computer Vision: Algorithms and Applications* by Szeliski remains an excellent reference (Szeliski 2022).

The excursion on adversarial examples touches on a topic that has become important both theoretically and practically. A standard early reference is the paper by Szegedy and colleagues, which first drew widespread attention to the existence of adversarial examples in neural networks (Szegedy et al. 2014). A particularly influential and accessible follow-up is the linear explanation by Goodfellow, Shlens, and Szegedy (Goodfellow et al. 2015).

As in the previous chapter, the best way to use these references is often to revisit them after gaining some hands-on experience. Once the basic ideas of convolution, feature maps, and residual connections have become familiar in practice, many of these texts become much easier to appreciate.

Appendix: PyTorch Essentials for NumPy Users

1. NumPy Interoperability

PyTorch tensors can share memory with NumPy arrays. Use `torch.from_numpy()` to create a tensor that points to the same memory as the array. Changing one will change the other.

```
import torch
import numpy as np

# Shares memory
np_arr = np.array([1, 2, 3])
t = torch.from_numpy(np_arr)

# Independent copy
t_copy = torch.tensor(np_arr)
```

2. Device Management (CPU vs. GPU)

Tensors must be on the same device to interact. Use `.to(device)` to move data.

```
device = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.randn(3, 3).to(device)
```

3. Understanding the Computational Graph

When you perform operations on tensors, PyTorch builds a Directed Acyclic Graph (DAG). Only tensors with `requires_grad=True` track operations to build a computational graph. Input data usually has this set to `False`, while model parameters have it `True`.

```
x = torch.ones(2, 2, requires_grad=True)
```

4. The Necessity of `detach()`

When converting a tensor that is part of a graph back to NumPy, you must `detach()` it first to strip the gradient history.

```
# Fails if requires_grad=True
# arr = x.numpy()

# Correct way
arr = x.detach().cpu().numpy()
```

5. Copying vs. Cloning

`clone()` creates a copy that remains part of the computational graph (gradients flow through it). `detach()` removes it from the graph. Often, you want both: `x.detach().clone()`.

```
y = x.clone() # Still tracked by autograd
```

- `detach()` breaks the link to the computational graph. It ensures that any future operations on the new tensor won't trigger gradient calculations back to the original source.
- `clone()` allocates new memory. It ensures that if you modify the values of the new tensor (in-place), the original tensor remains unchanged.

6. The Power of In-place Operations (_)

Methods ending with an underscore (e.g., `.add_()`) modify the tensor in-place. This is memory-efficient but can break the computational graph if that tensor is needed for gradient calculations.

```
x = torch.ones(2, 2)
x.add_(5) # x is now 6s
```

7. The .backward() Engine

The `.backward()` method is the trigger for PyTorch's Autograd engine. When you call it on a scalar (usually your loss), it calculates the gradient of that value with respect to every "leaf" tensor in the graph that has `requires_grad=True`. The resulting derivatives are stored in the .grad attribute of each tensor.

```
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = (x**2).sum()  # y = x1^2 + x2^2 (a scalar)

# Compute gradients
y.backward()

# dy/dx = 2*x
print(x.grad)     # Output: tensor([4.0, 6.0])
```

Scalar Requirement: By default, `.backward()` only works on a single value (a 0-dimensional tensor). If your output is a vector, you must first reduce it (e.g., using `.mean()` or `.sum()`).

8. The `zero_grad()` Necessity

Gradients are **accumulated** (added) in the `.grad` attribute every time you call `.backward()`. You must explicitly clear them before the next iteration.

```
# For models:
optimizer.zero_grad()

# For manual tensor gradients:
if x.grad is not None:
    x.grad.zero_()
```

9. Context Management with `torch.no_grad()`

During inference (validation/testing), use the `with torch.no_grad():` context manager. This disables gradient tracking, reducing memory usage and speeding up computations.

```
with torch.no_grad():
    prediction = model(input_data)
```

10. Checking `x.grad is not None`

In PyTorch, a tensor's `.grad` attribute is `None` until `.backward()` is called for the first time. Code that manipulates gradients manually must check for existence to avoid `AttributeError`.

```
if x.grad is not None:
    # Safe to perform operations
    print(x.grad)
```


Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. 2015. "Explaining and Harnessing Adversarial Examples." *International Conference on Learning Representations (ICLR)*.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. Adaptive Computation and Machine Learning Series. The MIT Press. http://www.deeplearningbook.org/.

He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–78. https://doi.org/10.1109/CVPR.2016.90.

Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "ImageNet Classification with Deep Convolutional Neural Networks." *Advances in Neural Information Processing Systems 25 (NeurIPS 2012)*, 1097–105.

LeCun, Yann, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE* 86 (11): 2278–324. https://doi.org/10.1109/5.726791.

Prince, Simon J. D. 2012. *Computer Vision: Models, Learning, and Inference*. Cambridge University Press.

Szegedy, Christian, Wojciech Zaremba, Ilya Sutskever, et al. 2014. "Intriguing Properties of Neural Networks." *International Conference on Learning Representations (ICLR)*.

Szeliski, Richard. 2022. *Computer Vision: Algorithms and Applications*. 2nd ed. Springer. https://doi.org/10.1007/978-3-030-34372-9.

Zhang, Aston, Zachary C. Lipton, Mu Li, and Alexander J. Smola. 2023. *Dive into Deep Learning*. Cambridge University Press. https://d2l.ai/.